课程笔记

CS336 第 8 讲：跨 GPU 通信与分布式训练原语

Codex 基于 Stanford CS336 2025 第 8 讲视频、字幕与官方课程材料重写

2026-04-07

视频作者/频道：Stanford Online
发布日期：2025 年 05 月 15 日
视频时长：1:15:10
视频链接：`https://www.youtube.com/watch?v=LHpr5ytssLo`

目录

1 从单 GPU 到多 GPU：这一讲想解决什么 **2**

2 Collective operations：分布式程序的语法 **2**

2.1 为什么课程把 collective operation 放在最前面 2

2.2 all-reduce 为什么经常被改写成 reduce-scatter + all-gather 3

2.3 本章小结 3

3 硬件拓扑决定了哪些通信模式是现实可行的 **3**

3.1 PCIe、NVLink、NVSwitch 与“层级化带宽” 3

3.2 为什么 NCCL 会成为实际训练里的关键组件 4

3.3 PyTorch distributed 的意义是什么 4

3.4 本章小结 4

4 把并行策略 concretize 成代码：data / tensor / pipeline 三种分片 **4**

4.1 课程为什么选择用深 MLP 演示分布式训练 4

4.2 Data parallel：切 batch，参数保持一致 4

4.3 Tensor parallel：切层内宽度，激活要频繁协同 5

4.4 Pipeline parallel：切层段，activation 顺着 stage 往前流 6

4.5 为什么真实系统里通常不止一种分片方式 7

4.6 本章小结 7

5 硬件层级不会消失，并行只是在更高层重演上一讲的故事 **8**

5.1 为什么课程最后说“we will always want bigger models” 8

5.2 本章小结 8

6 总结与延伸 **8**

1 从单 GPU 到多 GPU：这一讲想解决什么

第 7 讲已经讲清了分布式训练的几个大方向：data parallel、ZeRO、pipeline、tensor parallel。第 8 讲继续往前推进，但角度更“系统工程”一些。它不再只问“有哪些策略”，而是问：**跨 GPU / 跨节点通信的基本原语是什么，硬件拓扑会怎样限制这些原语，而这些原语又如何在实际训练代码里落地？**

课程开场就把统一主题说破：无论是在单 GPU 内部还是多 GPU 之间，真正难的都不是“算”，而是“数据离算子太远”。上周我们通过 fusion / tiling 减少显存访问；这周则是通过 replication / sharding / collective communication 去减少跨 GPU 与跨节点的数据搬运。

第 8 讲的压缩目标

把“分布式训练”从几类概念性并行策略，进一步压缩成通信原语、硬件拓扑和真实代码结构三层之间的映射关系。

2 Collective operations：分布式程序的语法

2.1 为什么课程把 collective operation 放在最前面

课程明确指出，collective operations 是现代分布式编程的概念基元。你可以把它们理解成“由一组 rank 共同参与、并且通信模式事先就被抽象好的大动作”。比起手工管理很多点对点消息，它们更适合大规模训练。

第 8 讲逐个回顾了 broadcast、scatter、gather、reduce、all-gather、reduce-scatter、all-reduce。这些名字如果只背定义其实意义不大；真正重要的是理解它们在训练里分别扮演什么角色：

- 广播参数或配置。
- 收集或拆分分片后的 tensor。
- 归约不同 rank 上的梯度或统计量。
- 在 all-reduce 与 reduce-scatter + all-gather 的等价关系下做更省带宽的实现。

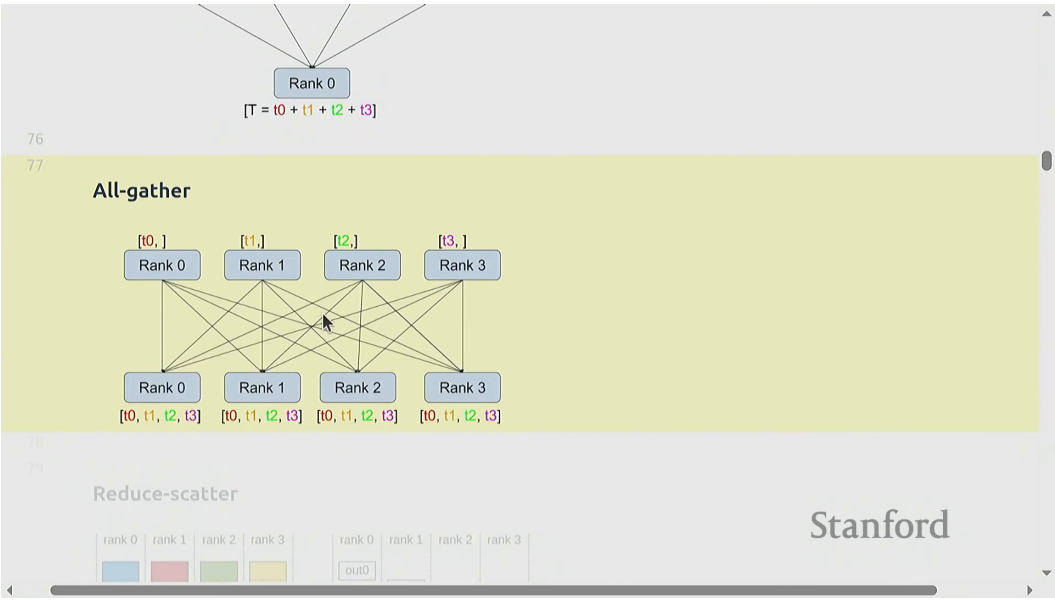

图 1: 课程用几种经典 collective primitive 先建立分布式训练的“语法层”。

2.2 all-reduce 为什么经常被改写成 reduce-scatter + all-gather

这一点在第 7 讲已经提过，但第 8 讲把它 concretize 成 code-level intuition。all-reduce 看似是“每个人都拿到全局归约结果”，但在带宽受限场景里，拆成 reduce-scatter 再 all-gather 往往更接近底层最优。原因在于：你先只拿自己负责的那段归约结果，再在必要时把各段聚回来，能减少中间步骤里的冗余传输。

为什么这条等价关系值得反复记忆

因为 ZeRO、FSDP 以及很多自定义分布式训练实现，都会不断利用“先分片归约、后按需聚合”这套逻辑。它不是实现细节，而是现代训练系统的中心结构之一。

2.3 本章小结

collective primitives 就像分布式训练的指令集。理解它们之后，后面的各种并行策略不再是黑箱框架名称，而是具体通信组合。

3 硬件拓扑决定了哪些通信模式是现实可行的

3.1 PCIe、NVLink、NVSwitch 与“层级化带宽”

课程这一讲特别强调，分布式训练必须服从硬件拓扑。单节点内 GPU 之间可能靠 NVLink 直接互连，多节点之间则可能走更慢的网络；不同互连层级的带宽和时延差异，会直接改变什么样的并行方案合理。

第 8 讲和第 5 讲其实在更高层上形成了呼应：单 GPU 里有 L1 / HBM / global memory 的 hierarchy；多 GPU 场景则继续出现 node 内、node 间、交换结构和 NIC 的 hierarchy。于是“靠近算子更快、跨层移动更贵”这条原则，从单卡一直延续到了整个机群。

关于分布式训练最危险的简化

不要把“多 GPU”理解成统一资源池。不同 GPU 之间的带宽差异巨大；同样一个 collective，在 node 内和 node 间的代价可能完全不同。

3.2 为什么 NCCL 会成为实际训练里的关键组件

课程专门提到 NCCL，不是为了介绍一个库，而是为了说明：collective operation 的高层接口最终都要落到低层拓扑感知与 packet scheduling 上。NCCL 会探测 GPU 拓扑、选择路径、组织 CUDA kernel 去发收数据。也就是说，**分布式训练从来不只是 Python 代码逻辑，而是通信库和底层硬件共同决定的系统行为**。

3.3 PyTorch distributed 的意义是什么

PyTorch 的 ‘torch.distributed‘ 在这里扮演的是“把 collectives 包成可用编程接口”的角色。课程故意不一上来就讲更高级框架，而是从更基础的分布式接口讲起，因为它想让学生先看懂底层动作，再理解更高级封装究竟替你管理了什么复杂度。

3.4 本章小结

第 8 讲前半部分真正建立的是一种层级化通信思维：collective 并不是抽象数学操作，它们的成本要通过特定硬件互连才能兑现，而 NCCL / ‘torch.distributed‘ 则把这一层复杂性部分封装起来。

4 把并行策略 concretize 成代码：data / tensor / pipeline 三种分片

4.1 课程为什么选择用深 MLP 演示分布式训练

课程明确说，它用 deep MLP 而不是完整 Transformer 来演示不同并行策略，是因为 MLP 在很多语言模型里本来就是主要计算瓶颈之一，而且结构更容易看清楚。这种选择和前几讲的风格一致：先抓最能暴露系统本质的骨架，再逐步扩展到更复杂模型。

4.2 Data parallel：切 batch，参数保持一致

在代码层，data parallel 的含义很直接：每个 rank 拿数据的一部分，各自做前向反向，然后通过梯度 all-reduce 保持参数一致。课程强调的关键点是：loss 在本地不同、梯度同步后才一致、参数因此维持一致。这一套逻辑把“同一模型副本 + 不同数据切片”写得很清楚。

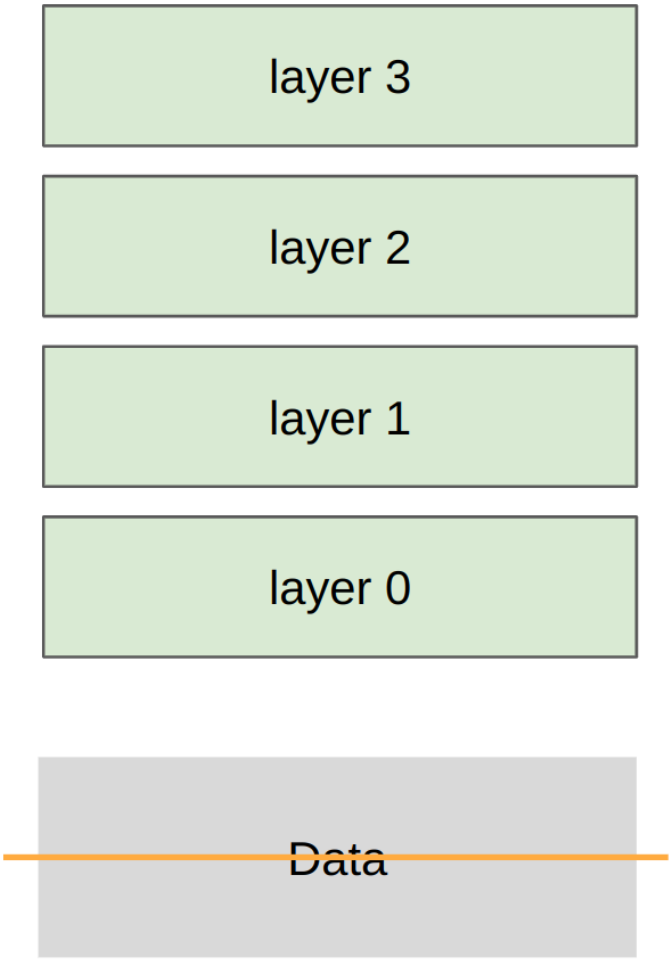

图 2: data parallel 的直觉：切的是 batch，不是模型本体。

4.3 Tensor parallel：切层内宽度，激活要频繁协同

tensor parallel 的关键不同，在于每个 rank 拿每层的一部分权重矩阵。你切开的是层内宽度，因此需要对 activations 做更细粒度的通信和同步。它的好处是没有 pipeline 的 bubble，坏处则是对高带宽互连依赖更强。

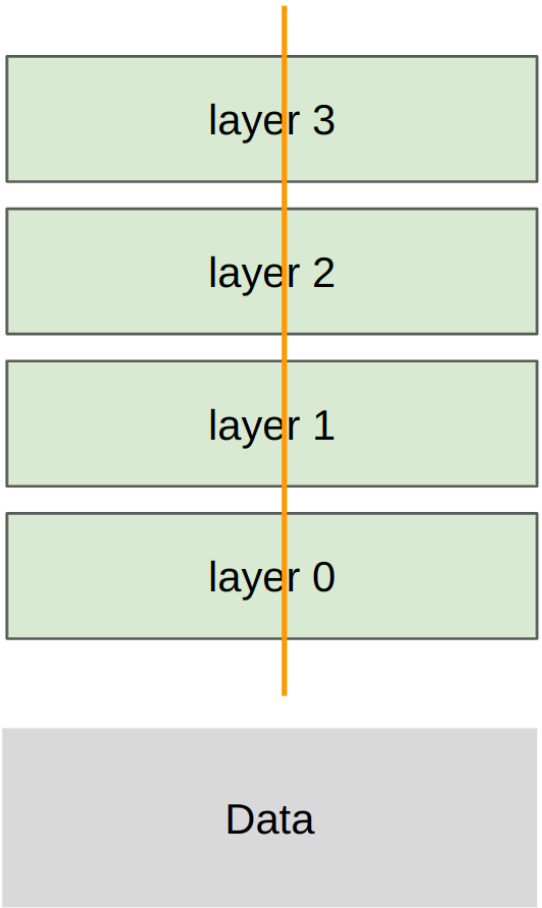

图 3: tensor parallel 切的是层内矩阵宽度，因此通信粒度更细，也更依赖快链路。

4.4 Pipeline parallel：切层段，activation 顺着 stage 往前流

pipeline parallel 的实现图景最容易想象：模型被切成若干连续 stage，每个 stage 在不同 rank 上。数据和 activation 在这些 stage 之间流动。课程也专门提醒：这里没有详细处理 overlapped communication / computation 以及复杂 pipeline schedule，但基础直觉已经足够清楚。

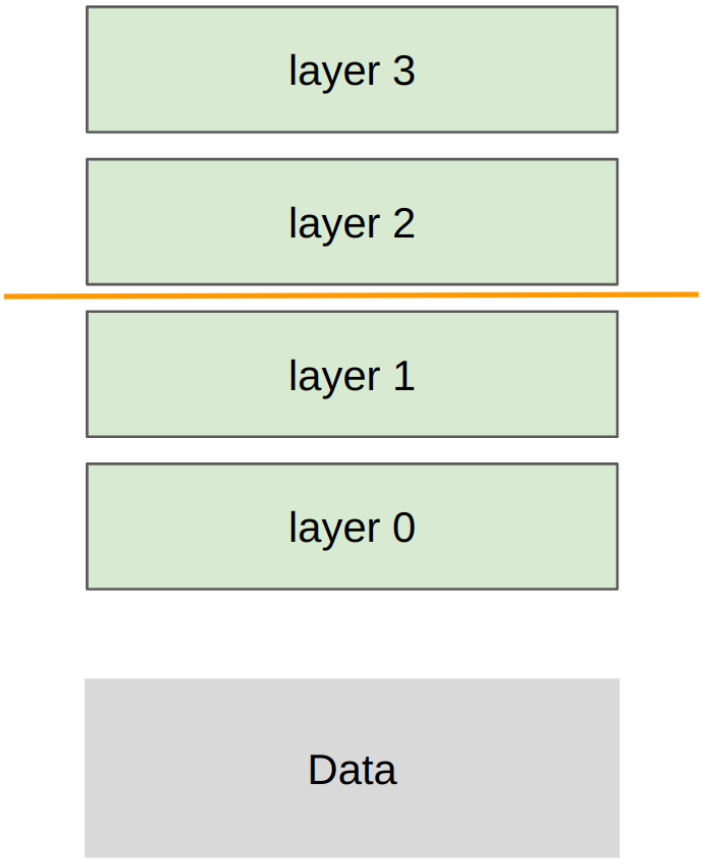

图 4: pipeline parallel 按深度切模型，本质是让 activation 在 stage 间流动。

4.5 为什么真实系统里通常不止一种分片方式

这一讲的一个关键结论是：这些策略不是互斥的。你完全可能在同一系统里同时使用 data parallel、tensor parallel 和 pipeline parallel。原因很简单：它们分别切 batch、宽度和深度，解决的不是同一个约束。

关于分布式训练的一个统一理解

切 batch、切宽度、切深度，本质上是在不同维度上重新分配“谁持有哪些参数、谁算哪些激活、谁承担哪些通信”。只要这样理解，很多具体框架实现都会变得更好读。

4.6 本章小结

第 8 讲这部分把并行策略从抽象设计变成了代码对象。data parallel、tensor parallel 和 pipeline parallel 的差别，本质上就是分片对象不同、通信对象不同、瓶颈位置也不同。

5 硬件层级不会消失，并行只是在更高层重演上一讲的故事

5.1 为什么课程最后说 “we will always want bigger models”

第 8 讲最后一句特别值得记住：硬件越来越快，但我们总会想训练更大的模型。因此，这种层级结构不会消失。换句话说，即便单卡更强、互连更快，工程师最终也还是会继续把问题推到更大的模型、更长序列、更高吞吐场景里，于是层级化 memory / communication 问题会一次次重演。

第 8 讲和第 5 讲之间最重要的呼应

第 5 讲讲的是单 GPU 上的 memory hierarchy 与数据搬运瓶颈；第 8 讲讲的是多 GPU / 多节点上的 communication hierarchy。它们其实是同一个故事在两个尺度上的重复。

5.2 本章小结

这一讲最值得带走的不是某一条 API，而是一个结构性理解：**并行训练从来不是把更多卡拼起来这么简单，而是把上一讲在单卡里发生的“数据远离计算”问题，重新放大到跨卡和跨节点层级**。

6 总结与延伸

第 8 讲最重要的结论可以压缩成四点。

第一，**collective communication 是分布式训练的语法**。训练系统的大量复杂行为，最终都能还原成几种 collective 及其组合。

第二，**硬件拓扑决定了通信现实**。 node 内、node 间、PCIe、NVLink、NVSwitch 的差异，会直接改变什么策略可行。

第三，**data / tensor / pipeline parallel 的差别，本质上是分片对象不同**。它们并不是名字差异，而是把数据、参数、激活分别切在不同维度。

第四，**并行训练不会“解决所有问题”，它只是把单卡上的 memory hierarchy 问题扩展成了更大尺度上的 communication hierarchy 问题**。

我对第 8 讲的压缩提醒

不要把分布式训练想成更大的 PyTorch 脚本。真正难的地方，不是多开几个进程，而是怎样在层级化硬件上安排通信、分片与同步，让系统既能跑，又尽量接近线性扩展。

到这里，parallelism 基础已经铺完。后面的 scaling、inference、data 和 alignment 章节都会把这种“资源层级 + 系统约束”的思想继续推进，只是关注对象从训练切换到别的模块。